Aman Garg

Senior AI Engineer | Data Scientist

+91-9309815072 | garg.aman96@gmail.com | LinkedIN | Digital Resume | Pune, India

Summary

AI Engineer / GenAI Developer / Data Scientist with 3+ years designing Large Language Model (LLM) systems end to end: Retrieval-Augmented Generation (RAG), GraphRAG, agentic tool-calling workflows and document extraction on Azure OpenAI, Vertex AI, LangChain, LangGraph and Neo4j, from retrieval design to evaluation and user-facing apps.

Technical Skills

GenAI: LLMs, RAG, GraphRAG, Agentic AI, Prompt Engineering, Natural Language Processing(NLP), RAG Evaluation
GenAI Frameworks: LangChain, LangGraph, LlamaIndex, OpenAI
Vector Databases: FAISS, ChromaDB, Azure AI Search, PG Vector
Data Science: Machine Learning, Deep Learning, Exploratory Data Analysis, Feature Engineering, Predictive Modeling
Languages & Tools: Python, SQL, Neo4j, Pandas, Streamlit, Git, Flask
Cloud & Platforms: Azure OpenAI, Azure AI Foundry, Azure ML Studio, Google Vertex AI

Experience

CitiusTech Healthcare Technology - Sr AI Engineer Oct 2023 – Sep 2026
Pune, India

Explainable Claims Denial Prediction | *Predictive Modelling, Azure OpenAI, Azure ML Studio, Python, Pandas, Neo4j*

- Explainable system that flags claims at risk of Authorization denial, with the reason behind each flag.
- Built a pattern engine: new claim → match against patterns in Neo4j → denial flag + rule behind it.
- Covered **3 Authorization sub-categories** (∼**4% of revenue cycle**); features picked from **400+ claim columns**.
- **LLM**-based feature discovery to surface denial drivers missed in domain review.
- PHI **guardrail**: PHI columns removed from claim data before any LLM call.
- Validated patterns on historical claims; generalized with DevOps into a reusable pipeline for all categories.

Conversational Analytics on Healthcare Knowledge Graph | *Neo4j, LangChain, Azure OpenAI, Tool Calling* **Impact Award**

- Natural-language analytics assistant that lets business users query an enterprise Neo4j knowledge graph.
- Built an **agentic tool-calling** workflow: question → query rewrite → schema fetch → Cypher → Neo4j → insight.
- Schema-grounded **Text-to-Cypher**: live graph schema injected into prompts to avoid hallucinated labels.
- Query rewriting to turn vague business questions into precise, graph-answerable queries.
- Self-correction loop: failed Cypher and its error fed back to the LLM for an automatic retry.
- Guardrails: read-only Cypher (no write/delete queries) and PHI fields excluded from responses.
- LLM-driven chart selection and auto-generated visuals; delivered as a self-serve Streamlit app.

Vector RAG vs. GraphRAG for HR Policy Q&A | *Vertex AI, Gemini, Neo4j LLM Graph Builder, LangChain, RAGAS*

- HR policy assistant over **100+ policy documents**, used to benchmark Vector RAG against GraphRAG.
- Built **Vector RAG**: overlapping chunks → embeddings → Vertex AI **Vector Search** → **reranking** → answer.
- Built **GraphRAG** with Neo4j LLM Graph Builder: **LLM-extracted entities and relationships** form the graph.
- **RAGAS-based evaluation** on answer correctness, faithfulness, relevance and hallucination.
- Vector RAG led on single-policy questions; GraphRAG led on **multi-hop**, cross-policy questions.

GenAI Document Intelligence | *Azure OpenAI, GPT-4o, LlamaIndex, Python*

- Field extraction system for insurance payor plan documents (SPDs), without per-payor model training.
- Built one **LLM + RAG extraction pipeline** that replaced **3 per-payor** Azure Document Intelligence models.
- Flow: **PyMuPDF** parsing → **hybrid search** (vector + BM25) → LLM answers field questions → **JSON**.
- Extracted ∼**100 fields per document** into a **fixed-schema JSON** for consistent downstream use.
- **Table-aware chunking**: tables kept whole during parsing so values stay with their headers.
- Reached **82% field-level accuracy**; a new payor needs only a new question set, with **no retraining**.

RAG-based Test Case Generation from Source Code | *Llama, FAISS, LangChain, Python* *RAG Architect*

- Local-LLM system that generates test cases directly from application source code and its documentation.
- Designed the end-to-end **RAG architecture**: parse → chunk → embed → store → retrieve → generate tests.
- **Code-aware chunking** at function and class level, so each chunk is a complete, testable unit.
- Stored **code embeddings** in **FAISS** with metadata (file, class, function signature) for precise lookup.
- **Context-aware retrieval**: target function plus related code and docs passed to the LLM to write tests.
- Led **model selection**; chose a **local Llama model** to keep proprietary code out of external APIs.

Education

Symbiosis Centre for Information Technology (SCIT) 2021 – 2023
MBA, Data Sciences & Data Analytics – Symbiosis International University *Pune, India*

Symbiosis Institute of Computer Studies and Research (SICSR) 2018 – 2021
Bachelor of Computer Applications – Symbiosis International University *Pune, India*